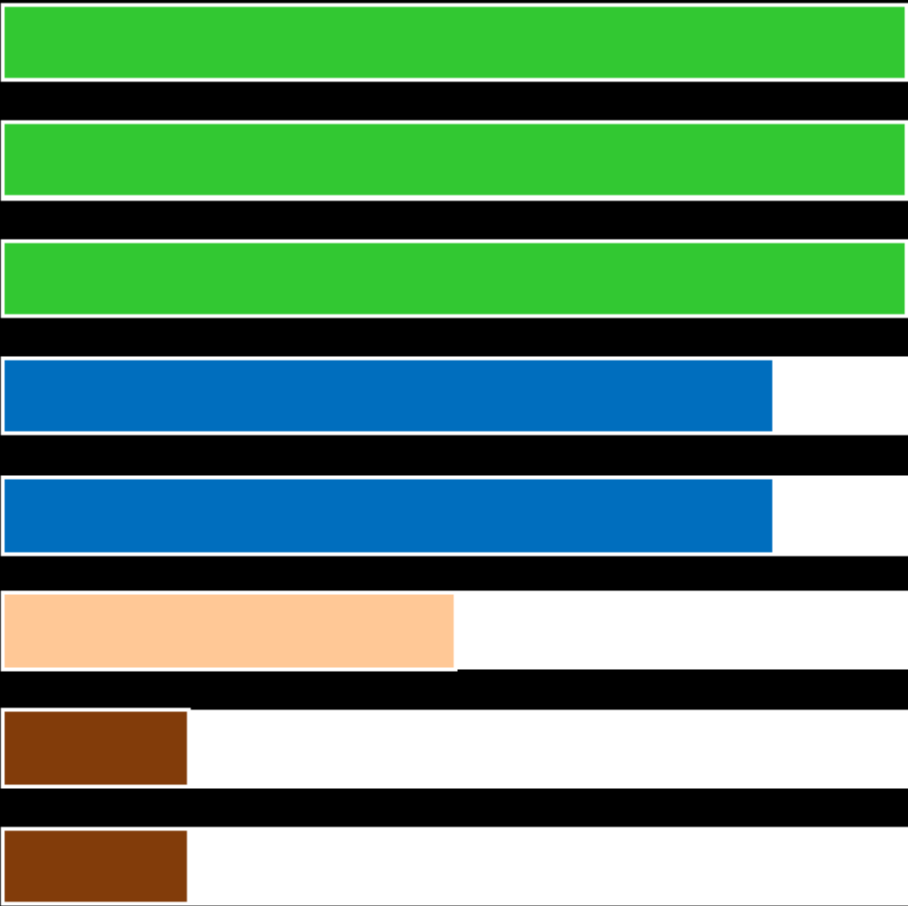




- IP Products:

- BCH Error Correction :
- CAN 2.0 Controller :
- Spacewire Controller :
- ONFI 2.1/2.2 Flash Controller :
- Gigabit Ethernet MAC :
- ONFI 4.2 Flash Controller :
- LDPC Error Correction :
- xSPI Controller :

Completion Ratio

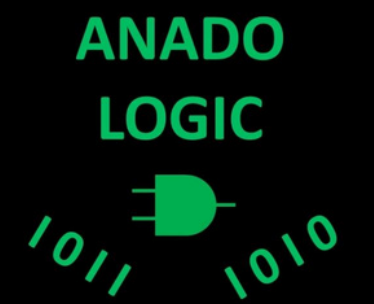




Real-Time Defect Detection with AMD Kria KR260

OUTLINE

What we'll cover today



1 | Problem

2 | Solution & System Overview

3 | Developing on Kria

4 | Inference

5 | Preprocessing

6 | Our Model

7 | Pipeline

8 | Results & Conclusion

PROBLEM

What are we solving?



The Challenge

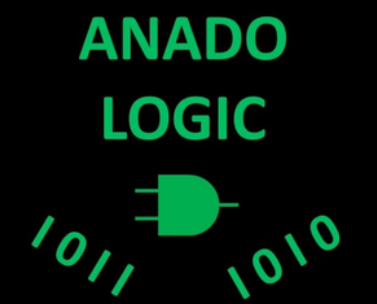
- Industrial lines need fast, accurate defect detection
- **CPU:** Bottleneck at preprocessing and AI inference
- **GPU:** High power + non-deterministic latency

Our Goal

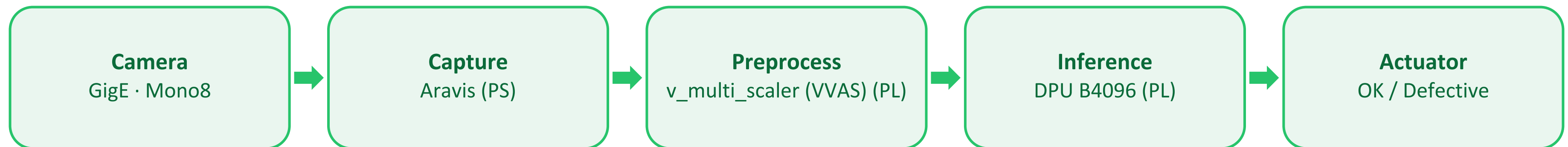
- **Fast & accurate detection:** FPGA-based pipeline with deterministic latency
- **No CPU bottleneck:** Hardware pre-processing + DPU inference fully offloaded to PL
- **No GPU power/latency issues:** Kria KR260 gives low power, real-time deterministic processing

SOLUTION OVERVIEW

End-to-end defect detection on KR260

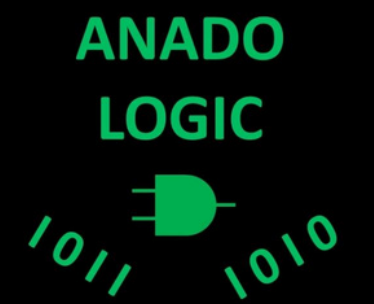


- **One board does it all:** capture on the PS, accelerate preprocessing and inference in the PL, then activate the actuator



SYSTEM OVERVIEW

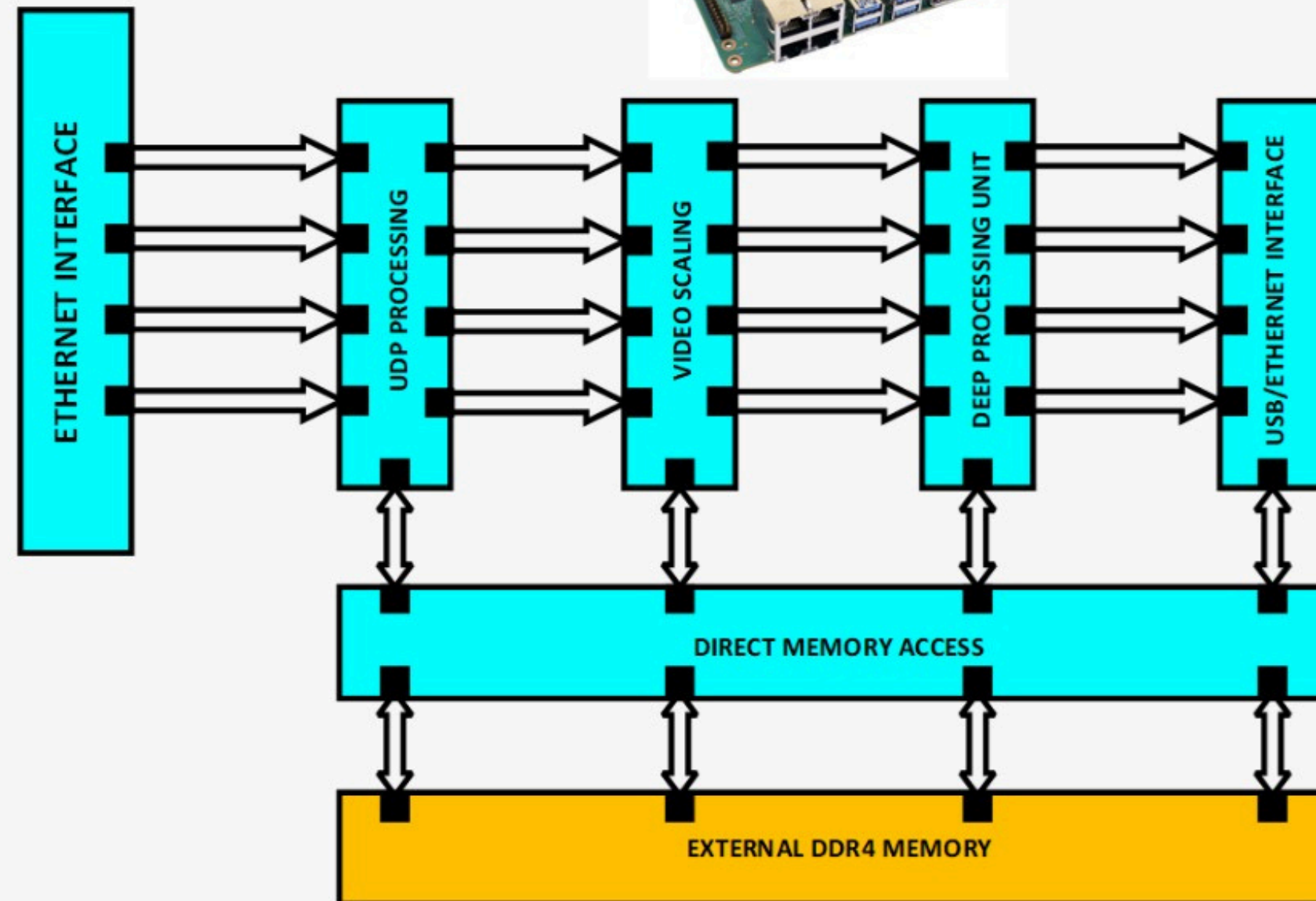
How the system works



GigE Vision Industrial Camera



KRIA KR260

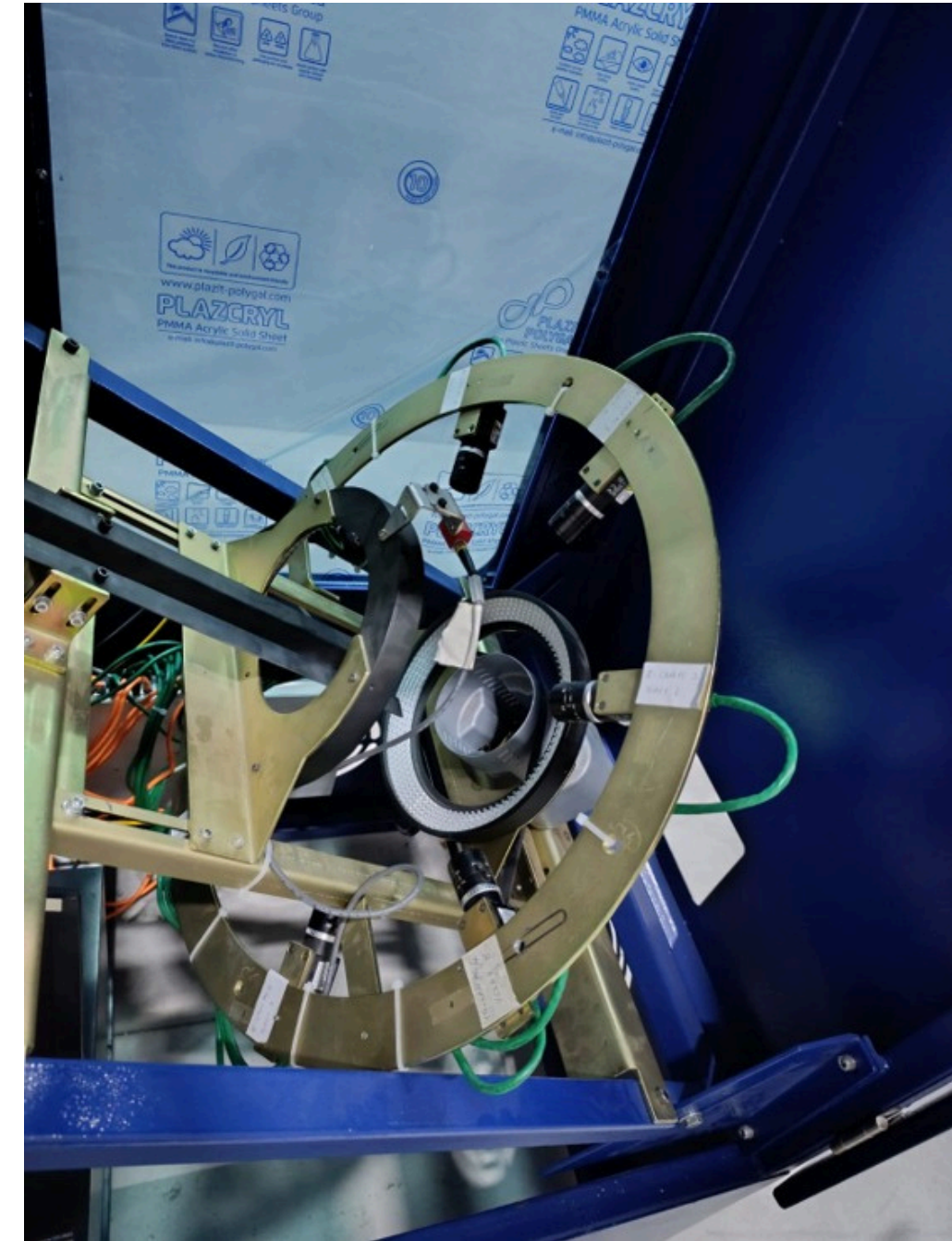
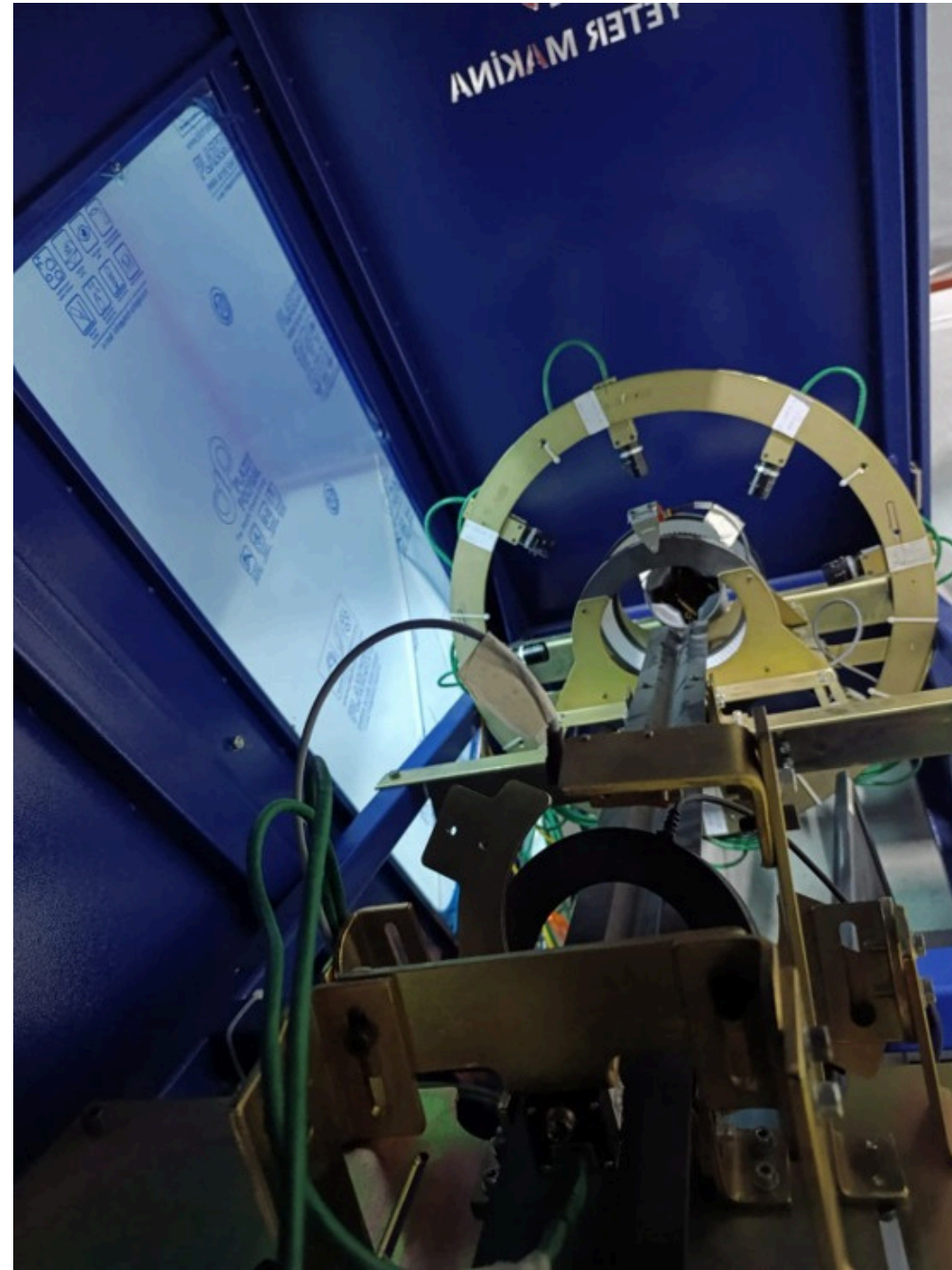
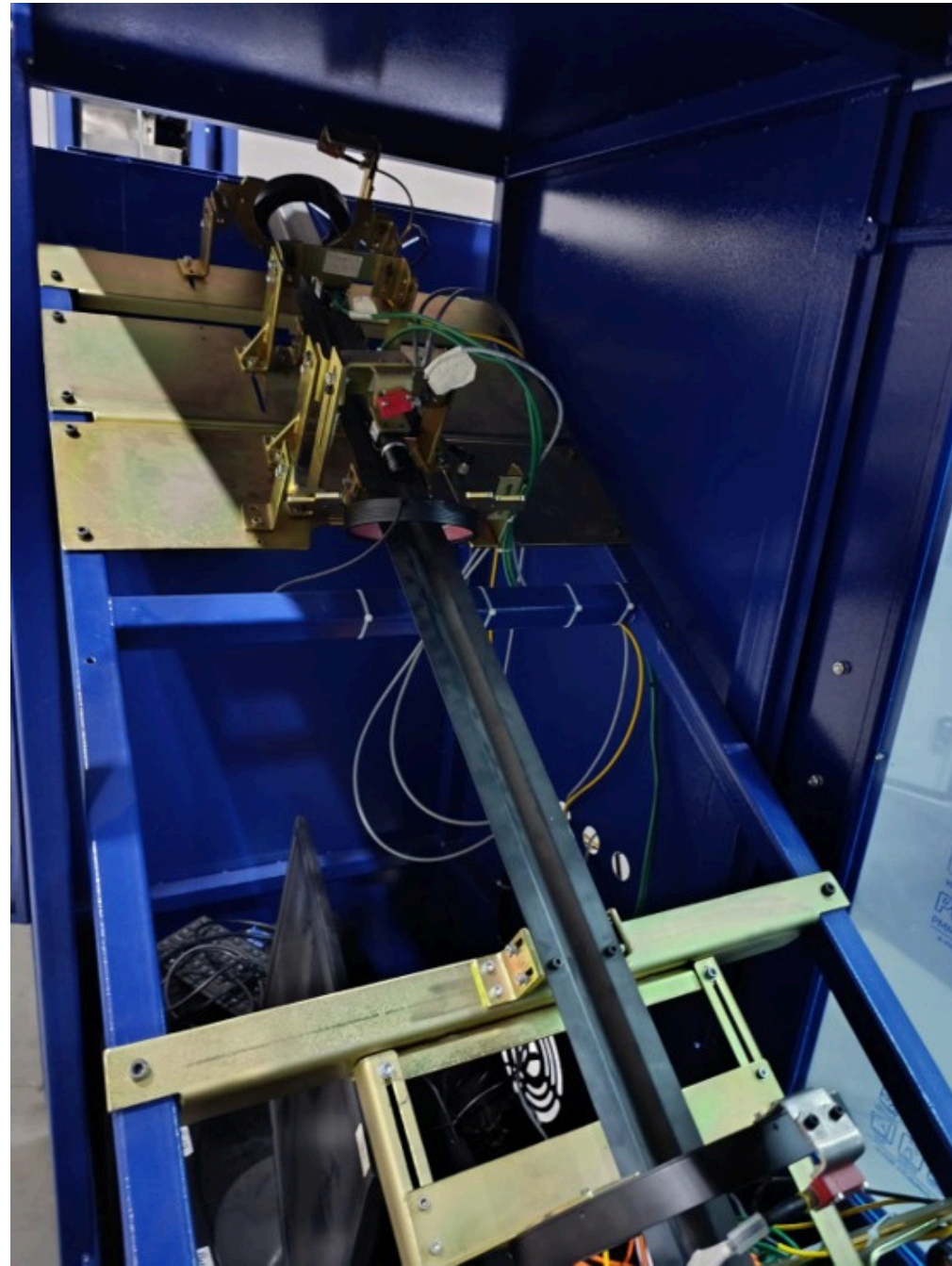
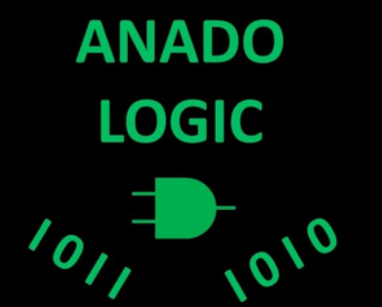


PLC



CARTRIDGE CASE INSPECTOR

Scratch, dent, crack inspection



Camera

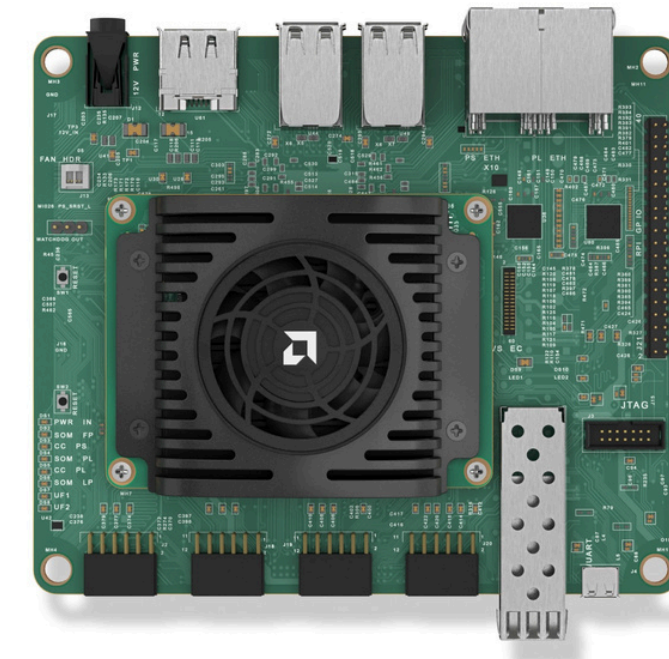
OMRON SENTECH STC-MBS202POE

- **Protocol:** GigE Vision 2.1
- **Pixel Format:** Mono8
- **Interface:** Gigabit Ethernet (PoE)



Processing Unit

- **SoM:** Kria K26
 - Zynq UltraScale+ MPSoC
 - 4× ARM Cortex-A53
 - FPGA fabric (PL)
- **Carrier Board:** KR260
 - Gigabit Ethernet, USB 3.0, DisplayPort
- **OS/Environment:** Ubuntu, PetaLinux

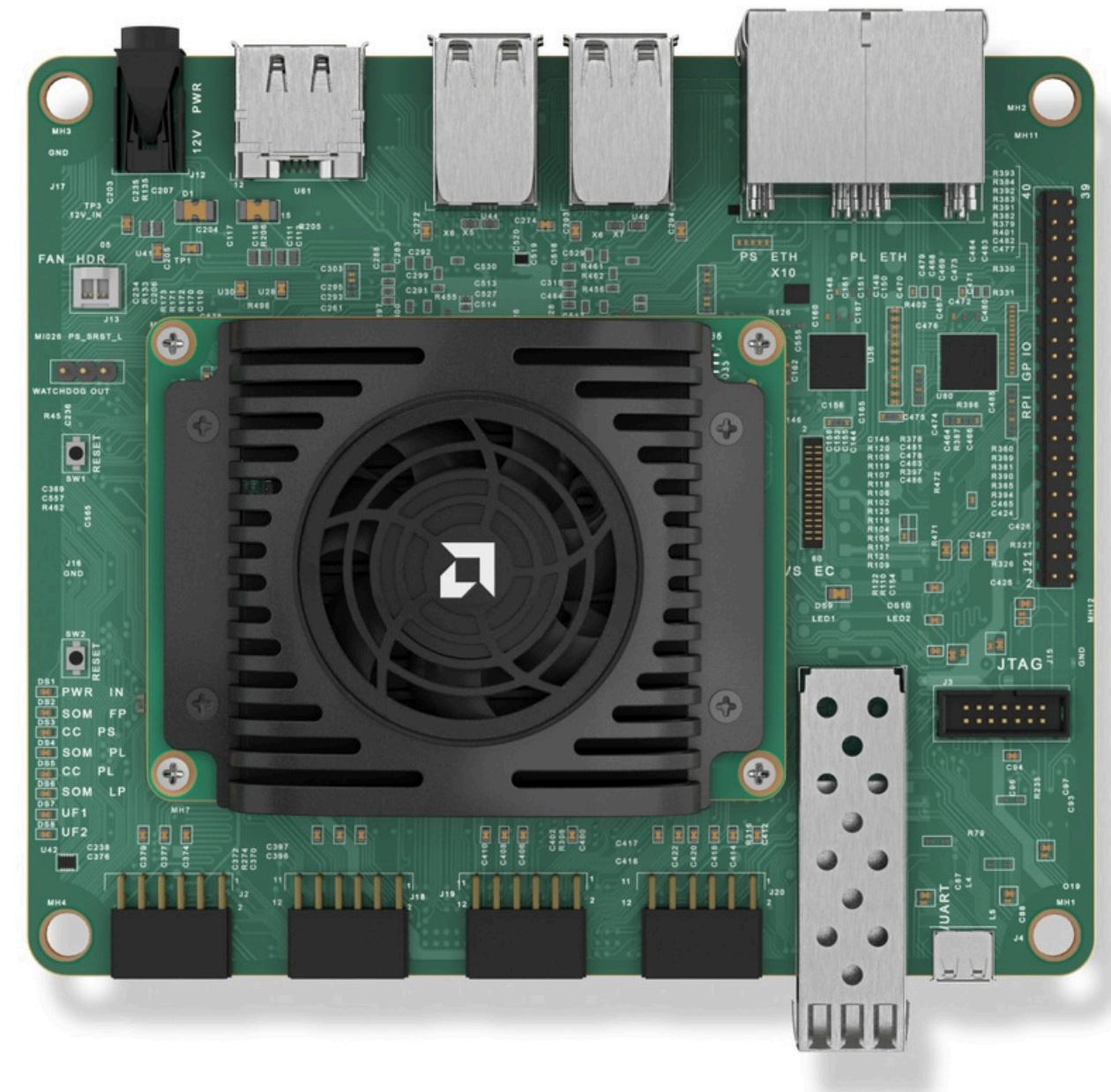


DEVELOPING ON KRIA

What is the Kria SOM?



- AMD Kria K26 SOM designed for edge AI, robotics and industrial applications
- Contains both ARM processors (PS) and FPGA fabric (PL), ZynqMP SoC
- Runs standard Ubuntu, no custom OS needed
- Low-power consumption, suitable for industrial environments



Kria KR260 Robotics Starter Kit

DEVELOPING ON KRIA

Bitstream Management: Legacy vs Kria SOM

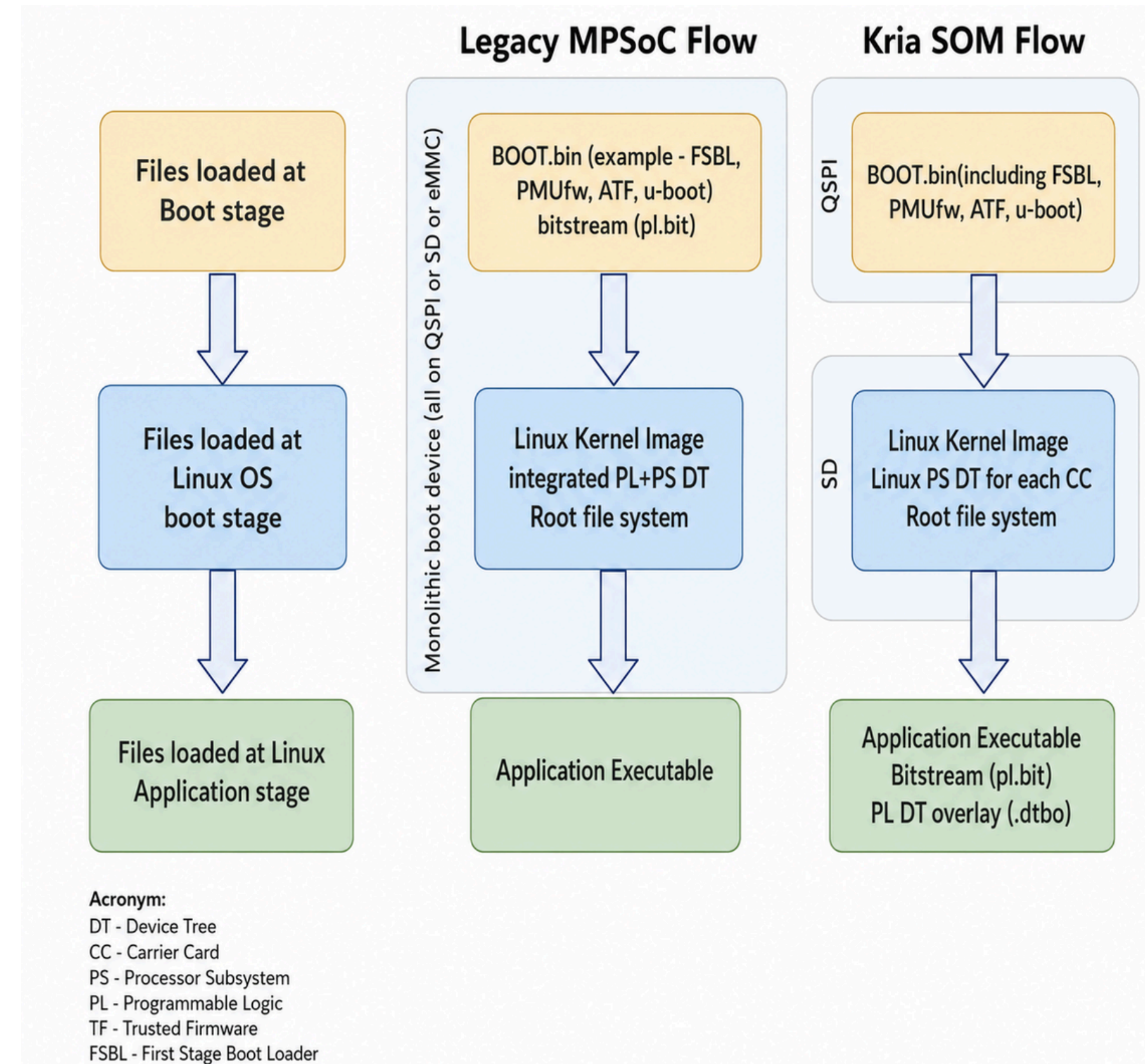


Legacy MPSoC Flow (static boot image flow)

- Bitstream baked into BOOT.bin and loaded at boot
- One OS per app
- Reboot to switch app

Kria SOM Flow (KV260, KR260)

- Bitstream not in boot image and loaded at runtime
- Single OS, multiple apps
- No reboot to switch app



Vivado flow

- Hand-built block design in IP Integrator
- You wire every IP (DPU, scaler, VDMA) manually
- Maximum control, most manual effort

Vitis flow (our choice)

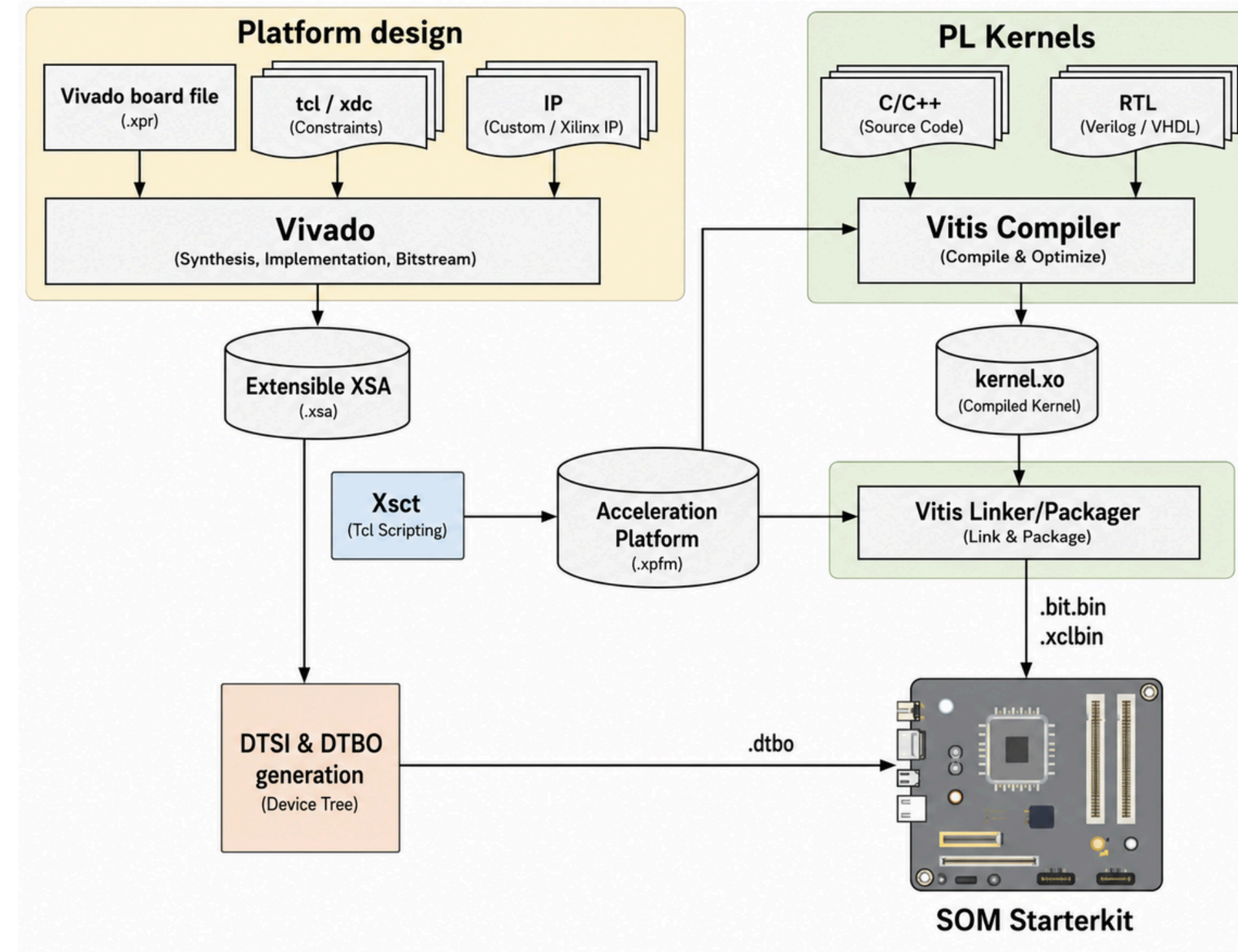
- Start from a base platform (AMD-provided or custom) + accelerator kernels
 - The tool links the platform + accelerator kernels (e.g. DPU, multiscaler) automatically, no manual wiring
 - Skips re-synthesizing the platform each iteration, only the kernel is synthesized
-
- **Bottom line:** we use the Vitis acceleration flow, less manual wiring and reusing the platform saves synthesis time

DEVELOPING ON KRIA

Vitis Acceleration Flow



- Platform + Accelerators → v++ --link → 3 output files
- **Platform (.xpfm)**: KR260 base hardware design, two options:
 - Use AMD provided platform (e.g. kv260_ispMipiRx_vcu_DP, kr260_pmod_gps)
 - Create custom platform with Vivado
- **Accelerators (.xo)**: PL kernels written in HLS or RTL
- **v++ --link**: combines platform + accelerators → generates .bit.bin + .xclbin + .dtbo



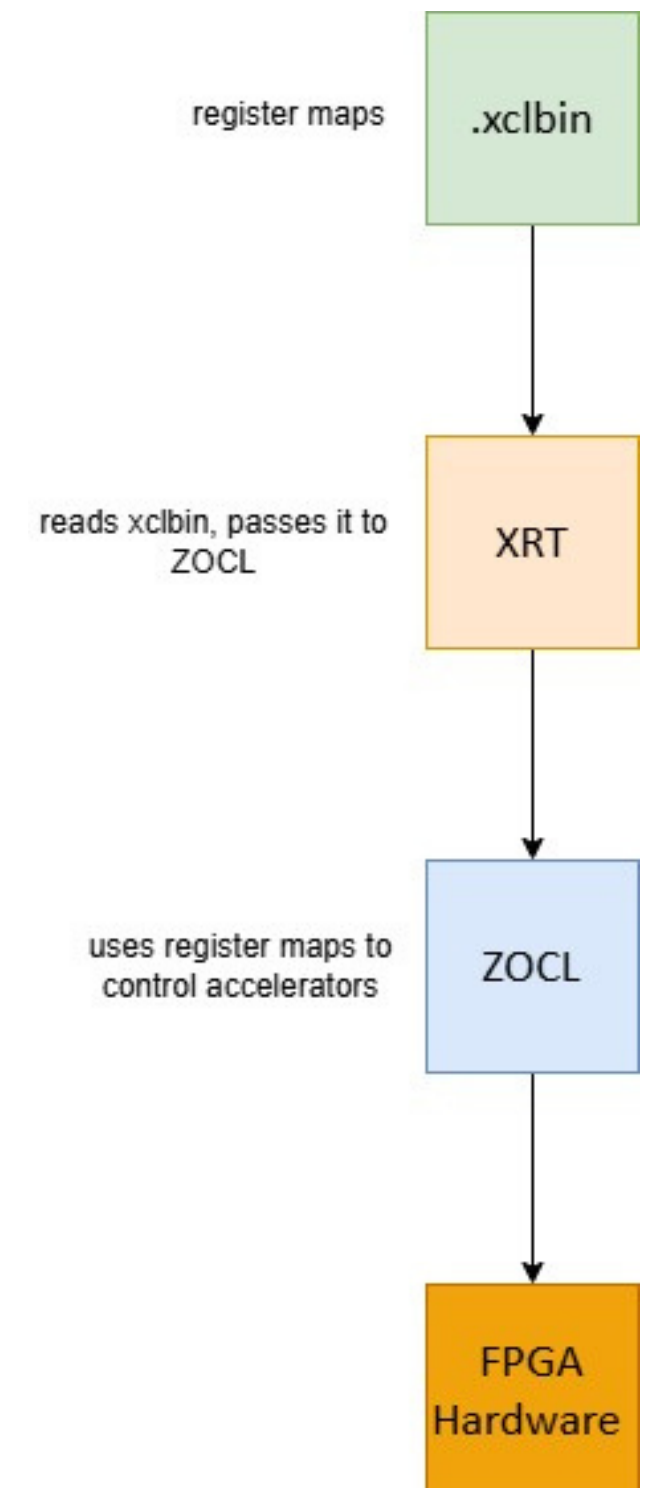
DEVELOPING ON KRIA

Runtime Software Stack: XRT & ZOCL



- **Load:** overlay is activated at runtime via the Linux FPGA manager (xmutil loadapp) without a reboot
 - **.bit.bin** → programs the PL fabric
 - **.dtbo** → tells Linux what hardware appeared in PL
- **Run:** once active, the application drives the accelerators through XRT + ZOCL
 - **XRT (Xilinx Runtime):** Software library running on PS (ARM) side that manages buffer allocation, DMA, kernel control
 - **ZOCL (Zynq OpenCL driver):** Linux kernel driver that physically maps/drives the PL registers
 - **.xclbin** → read by XRT for register maps + kernel metadata

Run Phase

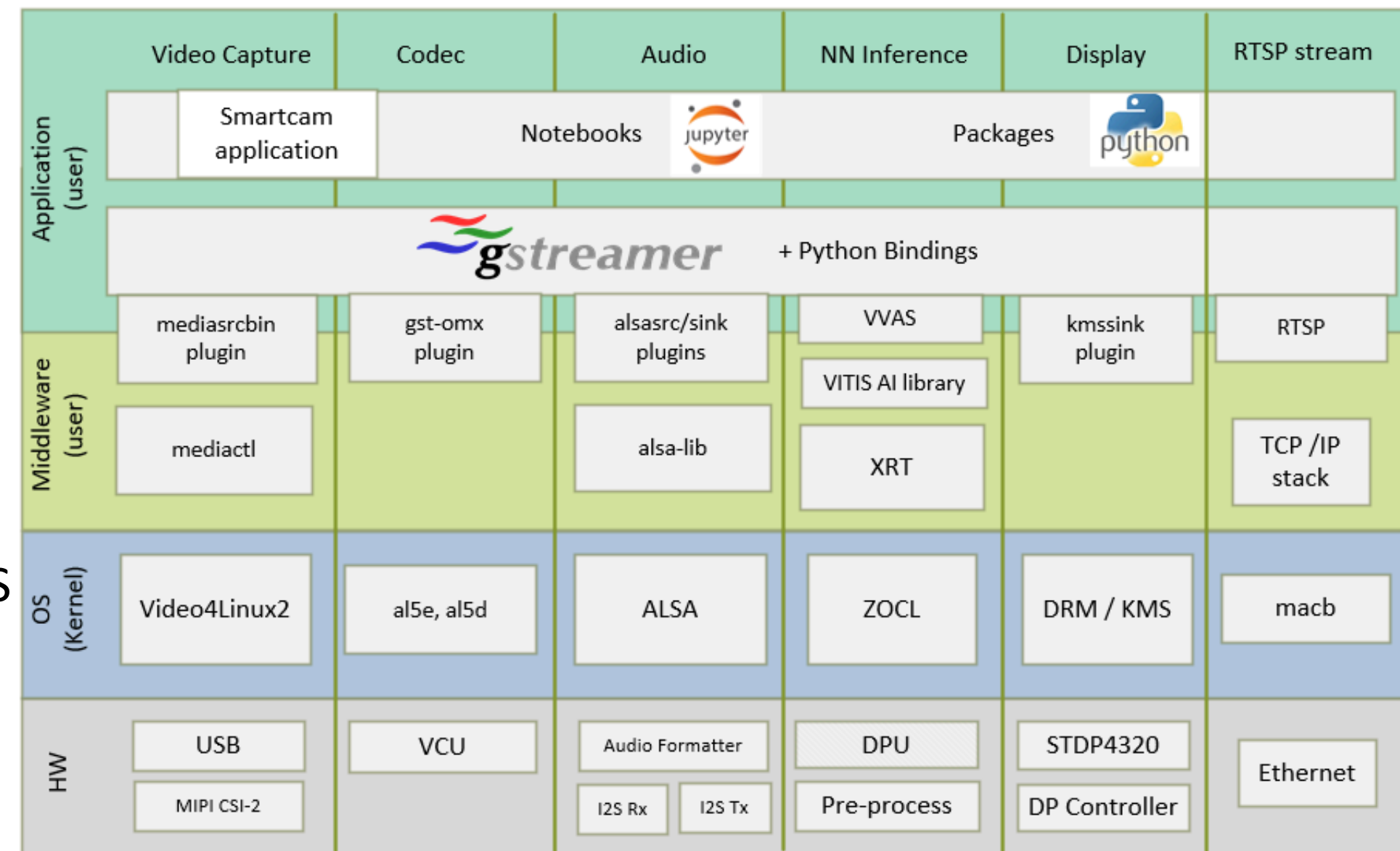


OUR IMPLEMENTATION

Custom FPGA overlay



- Custom FPGA overlay for Kria KR260 targeting real-time industrial defect detection
- **Camera:** OMRON SENTECH STC-MBS202POE — GigE Vision/ Mono8
- **Platform:** Minimal custom Vitis platform (clock/interrupt/AXI only, no extra fixed IP), built in Vivado
- **Accelerators:** DPU B4096 (Inference) + v_multi_scaler (HW preprocessing)
- **Software stack:** Ubuntu 22.04 / XRT 2.15 / Vitis AI 2.5 / VVAS 2.0 / GStreamer / Aravis
- **Build flow:** Vitis Acceleration Flow → v++ --link → .bit.bin + .xclbin + .dtbo



- **Output:** Binary defect classification — OK / Defective

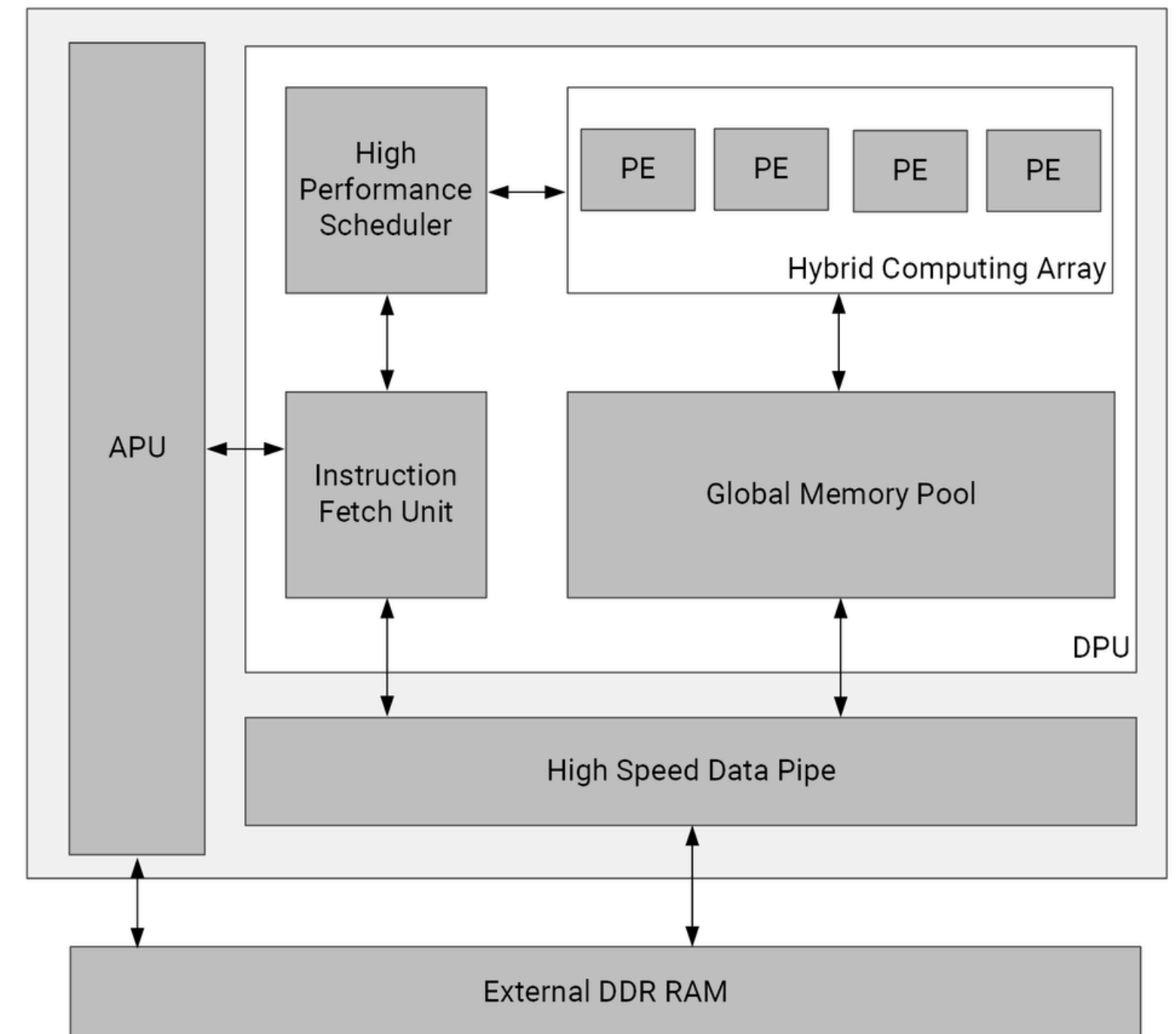
[Software Architecture of the Platform — Kria™ KV260 2022.1 documentation](#)

INFERENCE

What is the DPU?



- **Deep Learning Processing Unit:** Xilinx's hardware IP for running neural network inference on the FPGA
- Runs our custom-trained defect detection model directly in PL, no CPU involved
- Comes in different sizes (architectures), B512 up to B4096

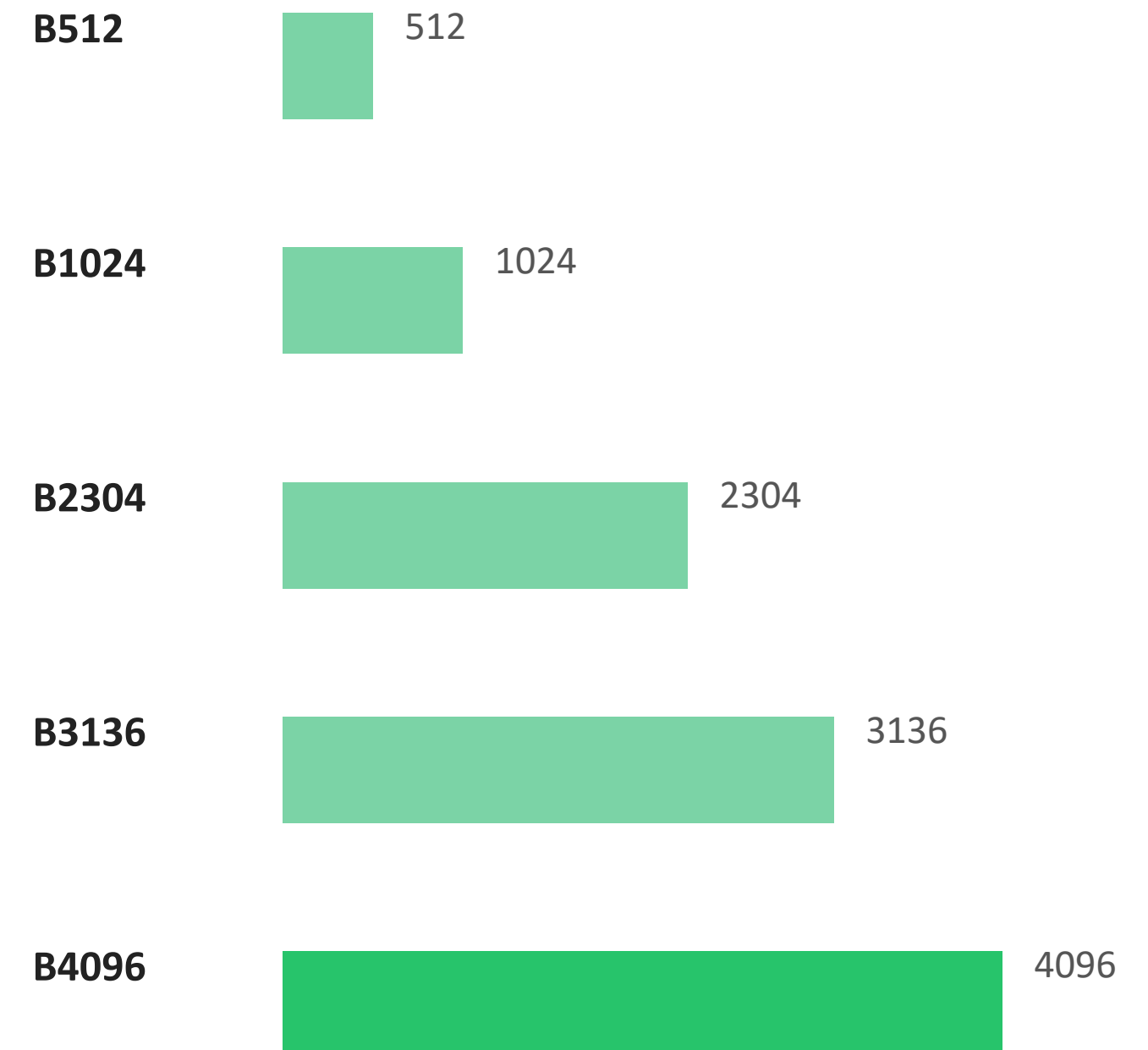


X22327-072219

What do B4096 and B3136 mean?

- The “B” number = **peak operations per clock** — what the convolution engine can do each cycle (1 MAC = 2 ops)
- **B4096 = 4096 ops/clock**; B3136 = 3136; B512 = 512. Bigger = more parallel MAC units
- **Set by three parallelism factors:** pixel (PP) × in-channel (ICP) × out-channel (OCP), × 2
 - B4096 = $8 \times 16 \times 16 \times 2$
 - B3136 = $8 \times 14 \times 14 \times 2$
- **More parallelism → higher throughput (more FPS)** but more LUT / FF / BRAM / DSP
- **A bigger DPU is faster but costs more PL resources** — the budget problem on the next slides

Peak operations per clock



- We want a powerful DPU like B3136 or B4096 but BRAM isn't enough. What can we do?

Table 1: Resources of Different DPUCZDX8G Architectures

DPUCZDX8G Architecture	LUT	Register	Block RAM	DSP
B512	26922	34543	72	118
B800	29721	41147	90	166
B1024	34074	48057	104	230
B1152	32169	47374	121	222
B1600	38418	58831	126	326
B2304	42127	68829	165	438
B3136	46714	79710	208	566
B4096	52161	98249	255	710

Table 2: PL Resources

Resource	K26 SOM	Description
System logic cells	256,200	Programmable logic cells for available
CLB flip-flops	234,240	Configurable logic block (CLB): Total number of flip-flops
CLB LUTs	117,120	Configurable logic block: Total number of look-up tables
Distributed RAM (Mb)	3.5	Distributed memory
Block RAM	144	Number of 36 Kb block RAMs
Block RAM (Mb)	5.1	Total block RAM memory footprint
UltraRAM blocks	64	288 Kb dual-port, 72-bit-wide memory with error correction
DSP slices	1,248	27 x 18 signed multiplier with 48-bit adder/accumulator
CMTs	4	Clock management tiles
Video Codec	1	H.264 and H.265 supported simultaneous encode/decode
HDIO	69	High-density I/O supports 1.2V to 3.3V rails
HPIO	116	High-performance I/O differential pairs supports 1.0V to 1.8V rails
GTH transceivers	4	Up to 12.5 Gb/s serial transceivers
Transceiver fractional PLLs	8	Specialized PLL used in high-speed serial communication transceivers
PCI Express	1	Controller for integrated PCIe that is in addition to the integrated controller for PCIe in the PS

Table 3: Resources of DPUCZDX8G using UltraRAM

- We can use URAM so that we can use a powerful DPU architecture
- Kria SOM has 64 URAM blocks available
- Configuring DPU to use URAM frees up BRAM for the scaler

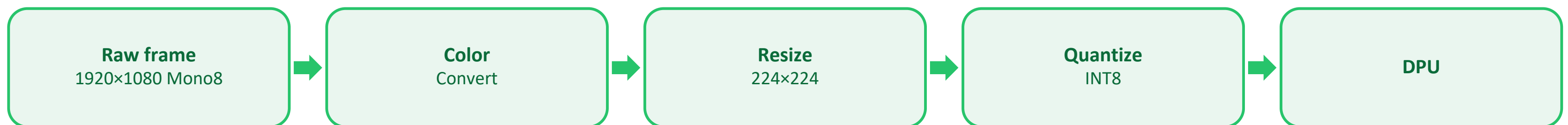
DPUCZDX8G Architecture	LUT	Register	Block RAM	UltraRAM	DSP
B512	26767	34538	0	18	118
B800	29578	41153	0	40	166
B1024	33752	48093	0	26	230
B1152	31901	47797	0	44	222
B1600	38197	58859	0	56	326
B2304	41736	69604	0	60	438
B3136	46260	80079	0	64	566
B4096	51843	98567	0	68	710

PRE-PROCESSING

Getting frames into the format the model expects



- **A trained CNN expects a fixed input tensor** — our ResNet-18 needs a set size, layout and value range, not the raw camera frame
- **Color space conversion:** the camera gives Mono8 (grayscale); the model needs a defined format and channel order (e.g. 3-channel BGR)
- **Scaling (resize):** the full frame (e.g. 1920×1080) is resized to the model input (e.g. 224×224)
- **Normalization / quantization:** pixel values are scaled to the INT8 range the DPU runs on
- **On the CPU this is a bottleneck** → we offload it to the v_multi_scaler in PL (next slide)

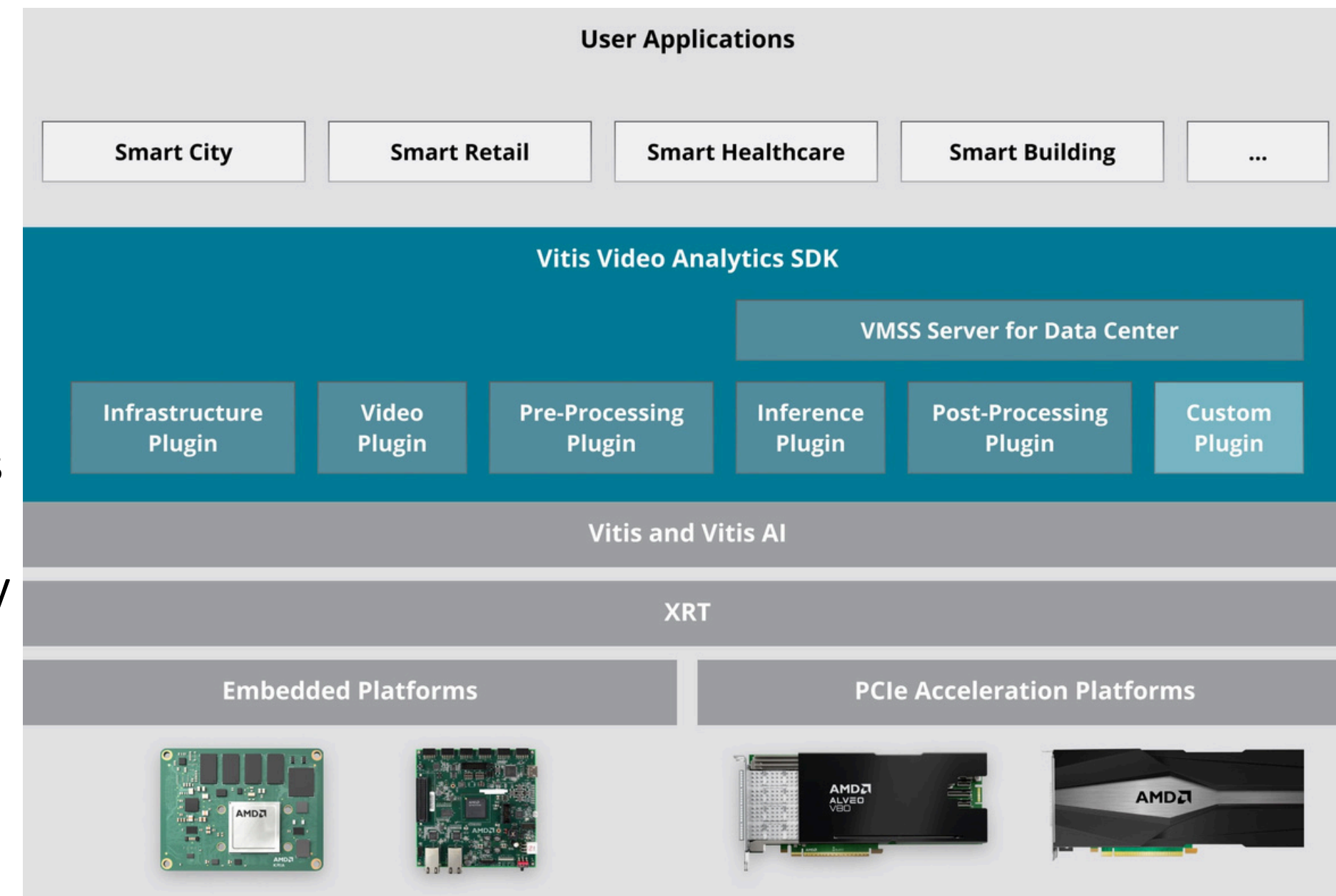


PRE-PROCESSING

What is VVAS / Multiscaler?



- **VVAS (Vitis Video Analytics SDK):** Xilinx's open-source framework providing GStreamer plugins and hardware kernels for video processing on FPGA
- **v_multi_scaler:** the HLS IP from VVAS that does hardware image preprocessing (resize, format conversion)
- Takes the load off the CPU, frees ARM cores for other tasks
- Inference runs through VART (Vitis AI Runtime), wrapped by the VVAS plugin



Classification

“What is in the image?”

One label for the whole image (e.g. OK / Defective)

► **Our task**

Detection

“What and where?”

A bounding box + a class for each object

Segmentation

“Which pixels?”

A per-pixel class mask - the exact object shape

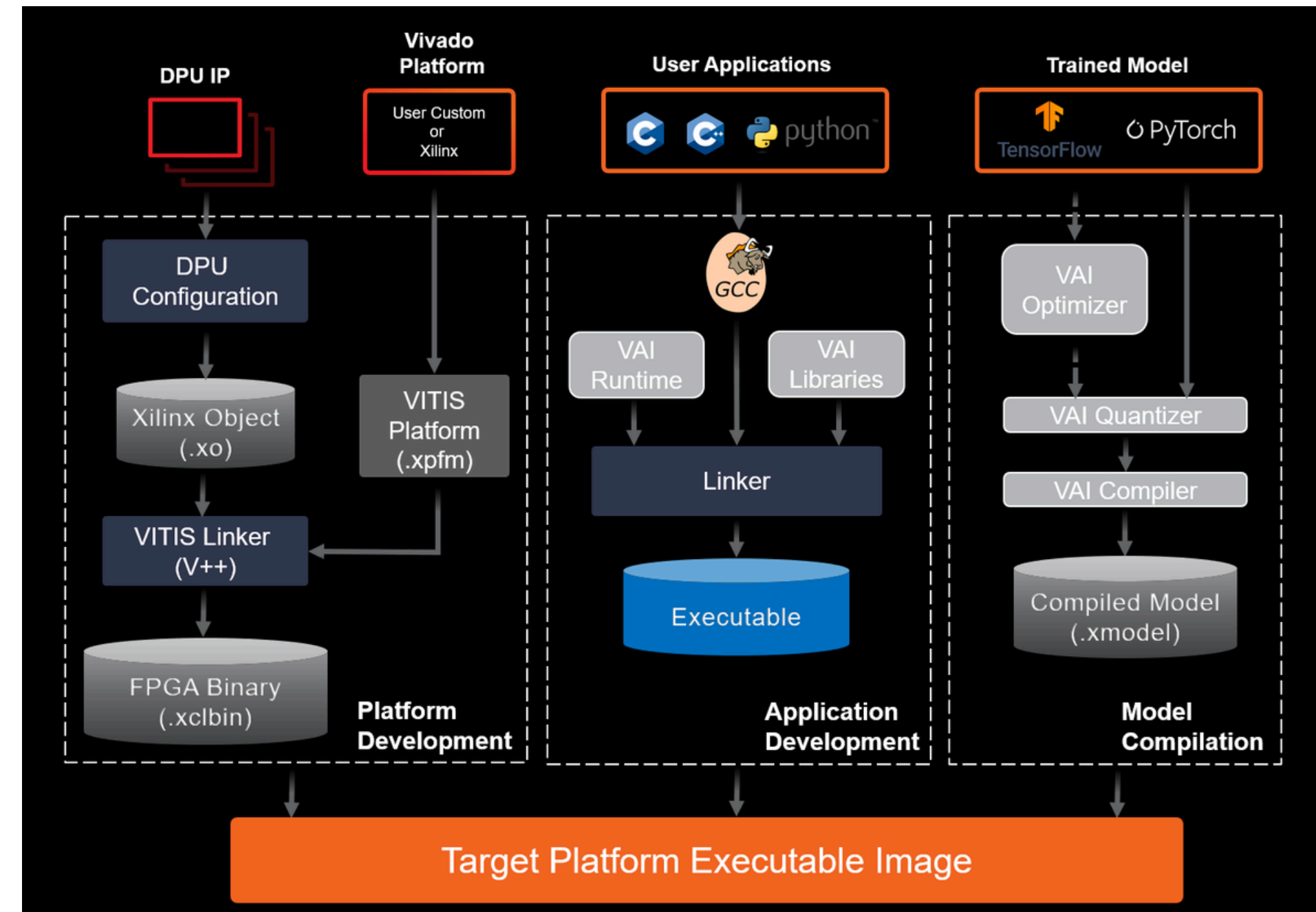
- **We do binary classification:** OK / Defective - one decision per frame

OUR MODEL

Custom defect detection, from PyTorch to the DPU

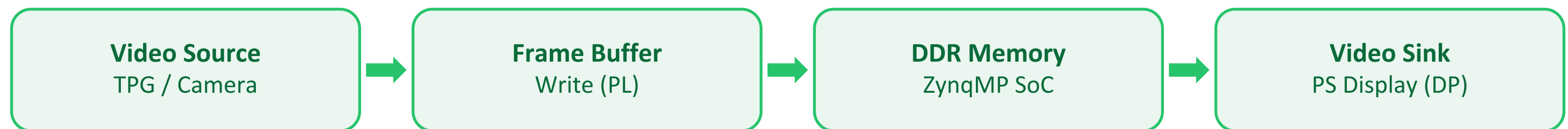


- **Model:** ResNet-18 (PyTorch)
 - **Binary classification:** OK / Defective
- **From model to FPGA, Vitis AI 2.5 flow:**
 - **Train:** ResNet-18 in PyTorch (FP32)
 - **Quantize:** Vitis AI 2.5 → INT8
 - **Compile:** Vitis AI compiler → .xmodel for the DPU
 - **Deploy:** Runs on DPU



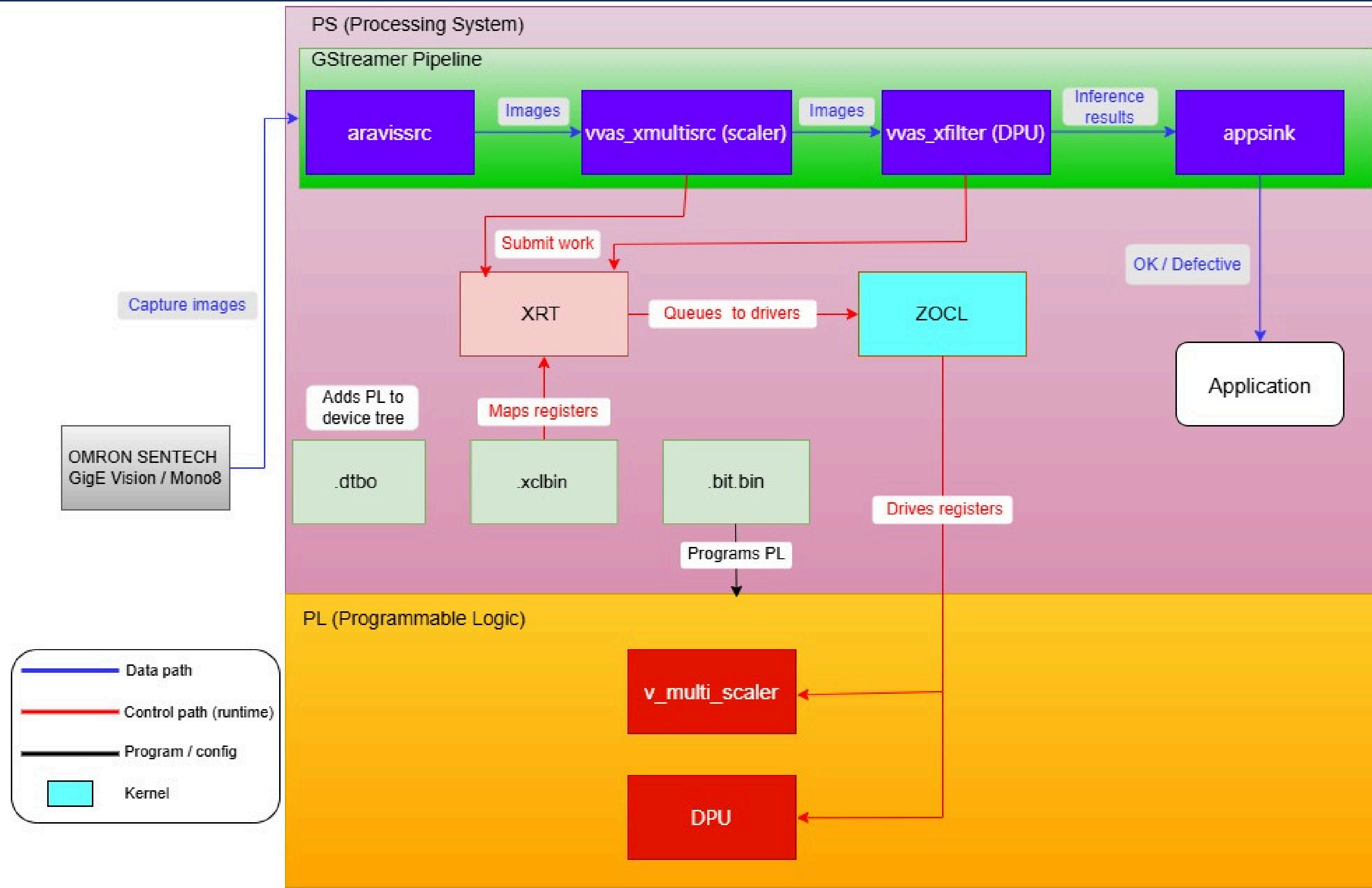
What is a video pipeline?

- **A processing chain** that moves frames from a video source to a video sink, one stage at a time
- **Video source:** where frames come from - a camera (GigE Vision), a Test Pattern Generator (TPG), or a file
- **Video sink:** where frames go - a display (DisplayPort / HDMI), an encoder, or memory
- **Connections:** AXI4-Stream carries the pixels between IPs; the frame-buffer-write moves them to DDR over AXI4 - no CPU in the data path
- **On the KR260** we bring the path up with a TPG first, then swap in the real camera



PIPELINE

Runtime architecture



Example GStreamer pipeline (VVAS)

```
gst-launch-1.0 -v \  
  aravissrc camera-name="Sentech-STC-MBS202POE" \  
  ! queue \  
  ! vvas_xmultisrc kconfig="/opt/xilinx/defect-detect/share/vvas/preprocess.json" \  
  ! video/x-raw,format=BGR,width=224,height=224 \  
  ! queue \  
  ! vvas_xfilter kernels-config="/opt/xilinx/defect-detect/share/vvas/aiinference.json" \  
  ! queue \  
  ! kmssink driver-name=xlnx fullscreen-overlay=true
```

- **aravissrc** — GStreamer source plugin from Aravis, an open-source GigE Vision driver; pulls Mono8 frames from the camera into the pipeline
- **vvas_xmultisrc** — HW preprocessing (resize + color convert) on the v_multi_scaler, config set via preprocess.json
- **vvas_xfilter** — runs the ResNet-18 .xmodel on the DPU via VART, config set via aiinference.json; attaches the OK / Defective result as metadata
- **kmssink** — displays the frame on DisplayPort (Xilinx DRM/KMS driver)

RESULT AND CONCLUSION

Resource Utilization



Dpu b4096 ve multiscaler

1. CLB Logic						
Site Type	Used	Fixed	Prohibited	Available	Util%	
CLB LUTs	63200	0	0	117120	53.96	
LUT as Logic	53932	0	0	117120	46.05	
LUT as Memory	9268	0	0	57600	16.09	
LUT as Distributed RAM	3840	0				
LUT as Shift Register	5428	0				
CLB Registers	114022	0	0	234240	48.68	
Register as Flip Flop	114022	0	0	234240	48.68	
Register as Latch	0	0	0	234240	0.00	
CARRY8	1718	0	0	14640	11.73	
F7 Muxes	2880	0	0	58560	4.92	
F8 Muxes	150	0	0	29280	0.51	
F9 Muxes	0	0	0	14640	0.00	

4. ARITHMETIC						
Site Type	Used	Fixed	Prohibited	Available	Util%	
DSPs	750	0	0	1248	60.10	
DSP48E2 only	750					

3. BLOCKRAM						
Site Type	Used	Fixed	Prohibited	Available	Util%	
Block RAM Tile	30.5	0	0	144	21.18	
RAMB36/FIFO*	23	0	0	144	15.97	
RAMB36E2 only	23					
RAMB18	15	0	0	288	5.21	
RAMB18E2 only	15					
URAM	64	0	0	64	100.00	

1. System Utilization							
Name	LUT	LUTAsMem	REG	BRAM	URAM	DSP	
Platform	1696 [1.45%]	135 [0.23%]	2404 [1.03%]	0 [0.00%]	0 [0.00%]	0 [0.00%]	
User Budget	115424 [100.00%]	57465 [100.00%]	231836 [100.00%]	144 [100.00%]	64 [100.00%]	1248 [100.00%]	
Used Resources	61510 [53.29%]	9133 [15.89%]	111618 [48.15%]	30 [20.83%]	64 [100.00%]	750 [60.10%]	
Unused Resources	53914 [46.71%]	48332 [84.11%]	120218 [51.85%]	114 [79.17%]	0 [0.00%]	498 [39.90%]	
DPUCZDX8G	50219 [43.51%]	6746 [11.74%]	99340 [42.85%]	15 [10.42%]	64 [100.00%]	710 [56.89%]	
DPUCZDX8G_1	50219 [43.51%]	6746 [11.74%]	99340 [42.85%]	15 [10.42%]	64 [100.00%]	710 [56.89%]	
v_multi_scaler	11291 [9.78%]	2387 [4.15%]	12278 [5.30%]	15 [10.42%]	0 [0.00%]	40 [3.21%]	
v_multi_scaler_1	11291 [9.78%]	2387 [4.15%]	12278 [5.30%]	15 [10.42%]	0 [0.00%]	40 [3.21%]	

RESULT AND CONCLUSION

Embedded World - 2026

